Module 5, Lecture 1 — Production Risk Management and Test Strategy

> **Course:** Staff Engineer / Technical Lead Mentorship **Module:** 5 — Risk register, testing strategy, security, performance, and quality gates **Lecture:** 1 of Module 5 **Case study:** "Spendwise," a personal finance application (FastAPI backend, PostgreSQL database, React Native client) **Format:** A one-way lecture. Everything is demonstrated for you, including the exercise, a review of a typical attempt, and a reference solution. Nothing here waits for your reply. **Builds on:** the PRD (Module 2), the Technical Design, ADRs, and API contracts (Module 3, Lecture 1, Lecture 2, and Lecture 4). Module 4 (work breakdown, estimation, sprint planning) has not been taught yet; nothing here depends on it, except that I use rough effort guesses, which Module 4 will make rigorous.

Part 1 — Learning objective

After this lecture you should be able to:

1. **Identify production risks** systematically, using several techniques instead of a brainstorm and a hunch.
2. **Score likelihood and impact** with scales anchored to your own product, and explain where a simple score misleads.
3. **Assign owners** properly, and write **mitigations, early-warning signals, and contingency plans** with triggers.
4. **Choose the test types** (unit, integration, end-to-end, security, performance, recovery) and explain **why each one exists,** which risk it reduces, and what it cannot catch.
5. **Define release gates** with measurable criteria, and say **what evidence** is required for each to pass.
6. **Trace** each important risk to a mitigation, a test, a gate, and a piece of evidence.

Part 2 — Real-world scenario

Three weeks before the pilot goes live, Meera sends a message:

> **Meera:** *"The app looks great! Are we ready? I told my friends it launches on the 30th. What could go wrong, and how do you know it won't?"*

Look at the second question: **"How do you know it won't?"** That is the question this whole lecture is about.

A junior developer answers: *"We've tested it, and it works."* An experienced lead hears five hidden questions inside that sentence and answers each:

Hidden question	What the lead has to be able to show
Tested against what?	A list of the specific things that could go wrong, ranked
How deeply?	Testing effort proportional to the risk, not to the lines of code
Tested how?	The right kind of test for each risk (a unit test cannot prove a backup restores)
What if it happens anyway?	A contingency plan with a trigger, rehearsed before it is needed
Who decides it is safe to launch, on what basis?	Release gates with objective criteria and stored evidence

Here is the situation, in numbers (all invented for teaching): the backend is feature-complete for the MVP; about ten pilot users will start, with perhaps a hundred or more if Meera's marketing works. The data is personal financial data. The team is one engineer. You have three weeks.

You cannot test everything. Nobody can. The skill is deciding **what to test, how hard, and what "good enough to launch" means,** and being able to defend those decisions in writing.

Part 3 — The lead's thought process

3.1 Risk, issue, and the idea that testing is risk reduction

Start with definitions, because people use these words loosely.

- A **risk** is something that **might** happen in the future and would affect the project or product. It has a **likelihood** (how probable) and an **impact** (how bad).
- An **issue** is a risk that **has already happened.** You manage it, you do not "score" it.
- A **test** is an activity that produces **evidence** about whether a risk is controlled.

Now the central idea of the lecture:

> **Testing exists to reduce risk by producing evidence.** A test that reduces no identified risk is waste. A significant risk with no mitigation, test, or contingency is unmanaged exposure.

This changes how you see a "test plan." It is not a list of features to click through. It is a **map from risks to evidence.** If you cannot say which risk a test addresses, ask why you are writing it. If you cannot say which test (or other control) addresses a risk, you have found a gap.

A well-formed risk statement has three parts: a cause, an event, and an impact:

> **Because** [cause or condition], **[event]** may happen, **resulting in** [impact on users, data, or the project].

Compare:

Weak	Strong
"Database problems."	"Because backups have never been restored, we may be unable to recover within 8 hours after data loss, resulting in an extended outage and possible loss of up to a day of users' expenses."
"Security."	"Because every query must remember to filter by user, a missed filter in any endpoint may expose one user's expenses to another, resulting in a privacy breach of financial data."
"Bugs."	"Because month boundaries use calendar dates in several queries, an off-by-one error may put expenses in the wrong month, resulting in wrong totals that users trust."

The strong versions tell you **what to do next:** restore a backup; write a two-user authorization test; test the 31st and the 1st. The weak ones tell you nothing.

DBA connection. You already think in risk. Every DBA has a mental list: *backup that cannot be restored, replication lag, disk full, a migration that locks a table, a runaway query, an unnoticed corruption.* Today we turn that instinct into a managed, written, scored, and tested list for the whole application.

3.2 How a lead identifies risks

A single brainstorm finds the risks you already worry about. A lead uses **several techniques** so that blind spots in one are covered by another.

Technique	How it works	What it tends to find
Pre-mortem	Imagine it is three months after launch and the project has *failed badly*. Each person writes down *why*.	Big, uncomfortable risks that people hesitate to say aloud
Walk the data flows	Take each important flow (save expense, sign-in, refresh, delete account) and ask at every step: *what can fail, and what does the user or data look like afterwards?*	Technical failure modes: retries, partial writes, races
Convert assumptions	Every item in the assumptions log is a risk: "users mostly online" becomes "if users are often offline, expenses are lost or delayed"	Product and requirement risks
Read your own documents	NFRs, ADR "negatives" and "revisit if," the failure tables in the Technical Design, accepted risks	Risks you already admitted in writing
Threat modelling	Assets, actors, scenarios (Module 3 Lecture 1 and Lecture 4)	Security and privacy risks
Dependency list	Everything outside your control: app stores, cloud provider, libraries, the client's accounts	External and schedule risks
Operational walk-through	Ask: who deploys, who monitors, who restores, who is on call, what if that person is ill?	People and operations risks
Category checklist	Run through product, technical, data, security/privacy, operational, delivery, external, people, compliance	Whatever the other techniques skipped
Experience and history	Known failure patterns; incidents from earlier projects	Recurring classics

What an inexperienced developer misses: they list **technical bugs** and nothing else. They miss the app store, the bus factor, the client's legal obligations, the scope creeping in, and the risk that the product works perfectly and *nobody uses it.*

What an experienced lead investigates: the risks that are **cheap to prevent early and catastrophic late,** and the risks that are **silent.** A crash is loud, so someone notices. *Wrong totals* are silent: the app keeps working while people make decisions on wrong numbers. Silent failures deserve extra attention, which I come back to in scoring.

3.3 Scoring likelihood and impact

A score exists to **rank risks so you spend effort on the right ones.** It is not mathematics; it is structured judgment. Two practices make it useful.

Practice 1: anchor the scales to your product. "Medium impact" means something different to everyone. So define each level in terms your client and your team would recognise. For Spendwise (and for the next three months of the pilot):

Likelihood (during the period under consideration):

Score	Label	Meaning
1	Rare	Less than about 5%
2	Unlikely	About 5% to 20%
3	Possible	About 20% to 50%
4	Likely	About 50% to 80%
5	Almost certain	More than about 80%, or it has already happened in testing

Impact:

Score	Label	Meaning for Spendwise
1	Negligible	Cosmetic. No effect on users' data or tasks
2	Minor	Annoyance with an easy workaround. No data affected
3	Moderate	A feature is unusable for some users for part of a day; or the schedule slips by up to a week. No wrong data shown
4	Major	**Wrong financial data shown to users;** or an outage longer than about 4 hours; or privacy exposure affecting a few users; or the schedule slips by more than a week
5	Severe	**Permanent data loss;** or exposure of many users' financial data; or breach of a legal obligation; or loss of the client's trust that ends the project

Score = likelihood × impact, read in bands:

Score	Band	Typical treatment
1 to 4	Low	Accept and monitor
5 to 9	Medium	Owner assigned; mitigate if cheap
10 to 14	High	Mitigation **and** contingency **and** tests before release
15 to 25	Critical	**Release blocker** until reduced

Practice 2: know where the arithmetic misleads, and correct for it.

1. **The "catastrophe" blind spot.** A risk with likelihood 1 and impact 5 scores only 5 (Medium). But "rare" is a guess, and "severe" means the project could end. **Rule: any risk with impact 5 receives at least High treatment regardless of its score,** or an explicit, owner-signed acceptance with a contingency.
2. **Silent failures.** A failure nobody would notice (wrong totals) is more dangerous than a loud failure of the same size, because it has no natural detection. **Rule: if there is no automatic detection, treat it one band higher,** or add detection as part of the mitigation.
3. **False precision.** The difference between a 9 and a 10 is noise. Don't argue over it. Argue about whether the *band* is right and whether the *scale definition* fits.
4. **Optimism and recency bias.** People underestimate risks they have not seen, and overestimate the last one that hurt. Ask: "What would make this a 4 instead of a 2?" and write down the answer. **Likelihood 5 includes "already happened in testing":** a failure you have *observed* is not rare.
5. **Scores change.** A risk before mitigation is the **inherent** score. After mitigation it is the **residual** score. Record both. A mitigation that does not change the residual score is not a mitigation.

An alternative method worth knowing: FMEA (Failure Mode and Effects Analysis) scores three things: *severity, occurrence, and detectability,* and multiplies them. Its explicit "detectability" factor handles silent failures well. For a small team the matrix plus the silent-failure rule is usually enough; FMEA becomes useful for safety-critical or complex systems. Context decides.

Risk appetite. How much risk is acceptable is not a technical question. It belongs to the **business owner** (Meera). Ten friends testing a pilot tolerate more risk than a public launch handling thousands of people's finances. A lead *describes* the risk in business terms and asks the owner to accept or reduce it. The lead does not accept business risk on the client's behalf.

3.4 Owners

A risk without an owner is a risk nobody will manage. An **owner** is the **single named person accountable for monitoring the risk and driving the response.** Not necessarily the one who does all the work.

Rules:

- **One name, never "the team."** If everyone owns it, no one does.
- **The owner has the authority or access to act,** or can escalate to someone who does. (A developer cannot own "the client's legal compliance.")
- **Distinguish the risk owner from action owners.** The owner of "backups may not restore" is the person accountable. The action "perform the restore drill" may be assigned to the same or another person, with a date.
- **Business risks belong to the client; technical risks to the lead.** In Spendwise: legal obligations, app store accounts, scope and budget decisions, and acceptance of residual business risk belong to Meera. Architecture, security controls, tests, backups, and deployments belong to the lead.
- **Every risk has a review date.** A risk reviewed never is a risk forgotten.
- **In a solo project you will own most technical risks,** which is itself a risk (the bus factor). Say so honestly in the register.

3.5 Responses: avoid, reduce, transfer, accept; mitigation versus contingency

Every risk gets one of **four responses:**

Response	Meaning	Example
Avoid	Remove the cause by changing the plan	Exclude bank import from v1 (the PRD already did this)
Reduce	Lower likelihood and/or impact	Make saves idempotent so retries cannot duplicate
Transfer	Move the consequence to someone better able to bear it	Use a managed database with provider-run backups; buy insurance; use a contract clause
Accept	Consciously live with it, recorded and owner-signed	Single API instance for the pilot

Two terms that are often confused:

- **Mitigation** is what you do **before** the event to reduce likelihood or impact. (Idempotency; two-user authorization tests; encrypted backups.)

- **Contingency** is what you do **after** the event happens, **if** mitigation was not enough. (Restore from backup; roll back the release; revoke all sessions and rotate the signing key.)

A good contingency has four parts:

1. **A trigger:** an observable fact that says "the risk has happened" (for example "error rate above 1% for ten minutes").
2. **An early-warning indicator:** something you can watch *before* the trigger (for example "connection pool usage above 80%").
3. **A written action plan:** steps, owner, and expected time.
4. **A rehearsal:** a contingency never tried is a hope. The restore drill is the classic example.

Residual risk is what is left after mitigation. If it is still High, you either do more, or the owner **explicitly accepts it in writing** (with an expiry date, so acceptance is revisited).

3.6 From risks to tests: why each test type exists

Now we connect risks to evidence. Each type of test is **good at catching some failures and blind to others.** A strategy is a *combination* chosen so that the blind spots of one are covered by another.

First, a word on **where tests run and what they cost.** Tests differ in speed, cost, and how often they give false alarms:

Level	Speed	Cost to write and maintain	Reliability (fewer false alarms)	Confidence per test
Unit	Milliseconds	Low	High	Low (small piece)
Integration	Seconds	Medium	Medium to high	High (real interactions)
End-to-end	Minutes	High	Lower (flaky)	Highest (whole path)

The classic **test pyramid** advises many unit tests, fewer integration tests, and very few end-to-end tests. For a **database-centred backend like ours, I adjust the shape:** the most valuable tests are **integration tests against a real PostgreSQL,** because most of our critical guarantees (constraints, `ON CONFLICT` idempotency, row locks, transactions, isolation, partial indexes) *only exist inside the database.* This is where your DBA background becomes a testing advantage. Some people call a heavier-integration shape a "testing trophy." Neither shape is a law. **Choose by where your risks live.**

Now each type, with its reason for existing:

Unit tests

- **What:** test one function or class in isolation, with no database and no network.
- **Why they exist:** to catch *logic errors cheaply and instantly*, and to document rules precisely. They are the fast feedback loop.
- **Spendwise examples:** money parsing and range checks; the budget threshold arithmetic in integers (`spent * 100 >= limit * 80`); month-boundary calculation; password policy rules; JWT claim verification logic (with a fake clock); error-code mapping; the retry/backoff calculation.
- **Techniques:** boundary values (exactly 80%, 79.99%, 100%, 0, the maximum); **property-based tests** (a library such as Hypothesis generates thousands of inputs and checks a rule, such as "parse then format returns the same amount"); a **fake clock** instead of real time.
- **What they cannot catch:** anything about real interactions: SQL mistakes, constraint behaviour, concurrency, configuration, deployment.
- **Warning:** *do not mock the database for logic that depends on database behaviour.* A mocked repository will happily "accept" a duplicate that PostgreSQL would reject.

Integration tests (API plus real PostgreSQL)

- **What:** run the FastAPI application against a **real PostgreSQL of the same major version as production** (commonly started in a container for the test run), and test through the HTTP interface and directly in the database.
- **Why they exist:** because the highest-value guarantees live at the boundaries: API to database, constraints, transactions, and the contract.
- **Spendwise examples:**
  - **Contract tests** from the API contract: every status code and error code, every validation rule (the twelve create-expense tests and sixteen auth tests from earlier lectures).
  - **Idempotency and concurrency:** two identical create requests at the same instant produce exactly one row and the right responses.
  - **Constraint backstops:** insert invalid data *directly into the database, bypassing the API*, and assert the database rejects it. This proves the safety net exists.
  - **Transaction behaviour:** a failure midway leaves no partial data; the failed-login counter **survives** a rolled-back-looking failure (the "commit despite error" rule).
  - **Authorization:** two users; each can never see, change, or delete the other's data (`404`).

    - **Migration tests:** apply all migrations from an empty database; apply the new migrations to a **snapshot of the previous release with seeded data**; check that the resulting schema matches what the code expects (a drift check).
- **What they cannot catch:** problems that appear only in the deployed environment (wrong configuration, certificates), at production data volumes, or in the mobile client.

Contract and schema-diff tests

- **What:** compare the generated OpenAPI document with the committed snapshot and fail on breaking changes; optionally generate many requests from the schema to look for crashes.
- **Why they exist:** to protect *old app versions* from silent breaking changes (a rename, a tightened rule).
- **What they cannot catch:** semantic changes the schema does not express.

End-to-end (E2E) tests

- **What:** exercise a real journey across the **deployed** system: for us, (a) API-level journey tests against **staging** (sign up, add an expense, see the summary, delete the account), and (b) a very small number of tests that drive the real mobile app (with a mobile automation tool such as Detox, Maestro, or Appium; verify current tooling).
- **Why they exist:** to catch **wiring and deployment problems** that no lower-level test sees: a missing environment variable, a wrong database URL, a migration not applied, an expired certificate, a misconfigured proxy, an app pointing at the wrong environment.
- **How many:** *few.* Three to five critical journeys. E2E tests are slow, break for unrelated reasons ("flaky"), and cost time to maintain. A hundred of them become a pipeline nobody trusts.
- **Also valuable: smoke tests** after every deployment: a tiny set (health check, sign in, create and read an expense) that answers "is this deployment alive and wired correctly?" in under a minute.
- **What they cannot catch:** subtle logic errors (too coarse), or rare concurrency bugs.

Security tests

- **What:** a mix of automated checks and focused manual review.
- **Why they exist:** because the cost of a security failure is high and the failure is often silent. Functional tests check what the system *does;* security tests check what it *refuses to do.*
- **Spendwise examples:**
    - **Authorization tests (broken object-level authorization):** user A requests, edits, and deletes user B's resources by ID → `404`, nothing changed.

    - **Authentication negative paths:** tampered, expired, `alg: none`, and replayed tokens; refresh reuse revokes the family; rate limits and the failure counter persist.
    - **Input abuse:** oversized bodies, unknown fields (`user_id` in a body), NUL characters, SQL-injection-style strings (which parameterised queries should neutralise).
    - **Dependency scanning:** a tool that checks Python and JavaScript packages against known vulnerability lists (for example `pip-audit`, `npm audit`).
    - **Static analysis and secrets scanning:** a Python security linter (for example Bandit) and a secret scanner on the repository and its history (for example gitleaks).
    - **A dynamic scan of staging:** an automated baseline scan (for example the OWASP ZAP baseline scan) to catch missing headers and obvious misconfiguration.
    - **Configuration checks:** TLS everywhere including the database connection, database roles without schema-change rights, debug mode off, no secrets in the mobile bundle.
    - **Log hygiene test:** capture logs during the test run and assert that no password, token, or hash appears.
    - **Manual:** a focused review of the threat model and the auth code by a second person if possible.
- **What they cannot catch:** a sophisticated, novel attack. Security tests reduce risk; they never prove safety. A lead says so.

Performance tests

- **What:** measure latency, throughput, errors, and resource use under load, against the NFRs (for us: p95 save ⩽ 500 ms at 50 concurrent users).
- **Why they exist:** because performance problems appear *only* at realistic volumes, and functional tests never show them.
- **The rules that make them valid:**
    1. **Use realistic data volume.** A query that is instant on 100 rows can take seconds on 3 million. Seed the database to the size you expect (for example the 1,000-user estimate from the Technical Design), and **refresh table statistics** before measuring.
    2. **Use a realistic user distribution.** Fifty simulated users who are *all the same user* hammer one set of rows and create artificial lock contention. Fifty *different* users do not.
    3. **Use a production-like environment:** same PostgreSQL major version, similar instance class, same connection-pool settings.
    4. **Measure percentiles,** not averages. A fine average hides a terrible tail.

5. **Watch the database while you test:** CPU, connections, lock waits, slow queries (`pg_stat_statements`), and the query plans (`EXPLAIN (ANALYZE, BUFFERS)`).

- **Types:** *load test* (expected load), *stress test* (find the breaking point), *spike test* (sudden burst), *soak test* (hours at moderate load, to expose connection leaks and slow growth).
- **Tools (examples):** k6, Locust.
- **What they cannot catch:** correctness bugs; and results are only as good as the realism of the data and environment.

Recovery tests

- **What:** deliberately rehearse the *contingency plans:* restore a backup, roll back a release, survive a dependency outage, rotate a secret.
- **Why they exist:** a contingency that has never been executed is an assumption. Recovery tests turn "we have backups" into "we **restored** a backup in 3 h 40 min and verified the data."
- **Spendwise examples:**
  - **Backup restore drill:** restore a recent backup into a *separate* database; verify row counts and checksums against the source; run the application's smoke tests against it; **record the elapsed time** (that is your *measured RTO*). Also test **point-in-time recovery** to a chosen timestamp (that proves your RPO).
  - **Application rollback drill:** deploy a deliberately bad release to staging and roll back; measure time.
  - **Migration rehearsal on a production-size copy:** record lock waits and duration.
  - **Dependency failure injection:** stop the database in staging; the API should return `503` cleanly and recover when the database returns, with no restart needed.
  - **Secret rotation drill:** rotate the signing key using `kid` and confirm no one is signed out.
  - **Deletion verification:** confirm that deleting an account removes data from live tables, and that backup retention matches the promise (NFR-8).
- **What they cannot catch:** disasters that differ from the rehearsed one. A rehearsal reduces surprise; it does not eliminate it.

Other tests that matter (shorter)

- **Usability test** for NFR-2 (a first-time user records an expense in under a minute): watch three real people.
- **Compatibility test:** the device and OS matrix agreed in the PRD.
- **Exploratory testing:** a time-boxed session ("charter") where a person tries to *break* the app guided by risks, not scripts. Finds what scripted tests never imagined.

- **Data-quality probes in production:** small scheduled queries that check invariants (no accounts pending deletion beyond 30 days; no expired tokens older than the purge window; no duplicate rows where uniqueness is expected). These are *tests that run forever,* and they catch the silent failures.
- **Testing the alerts:** deliberately trigger an alert to confirm that it reaches you. An untested alert is a hope.

3.7 Principles for designing the tests

1. **Depth proportional to risk.** High-risk areas (money, authorization, deletion, recovery) get layered tests at several levels. Low-risk areas (a label colour) get little or none.
2. **Test behaviour and invariants, not implementation.** "Two identical creates leave one row" survives refactoring. "Function X calls function Y" does not.
3. **Every test traces to a requirement or a risk ID.** Put the ID in the test name or a marker. A test with no trace is a candidate for deletion.
4. **Test the failure paths on purpose:** retries, timeouts, crashes mid-transaction (fault injection), invalid input, concurrency.
5. **Deterministic and isolated.** No dependence on the wall clock, test order, or shared state. Use a fake clock; give each test its own data.
6. **Real database, real constraints** for anything that depends on them.
7. **Never use real personal data in tests.** Generate synthetic data. If an anonymised production snapshot is ever needed, it needs the same care as production.
8. **Coverage is a signal, not a goal.** "90% line coverage" says code *ran,* not that anything was *checked.* A test with no assertion still gives coverage. Aim for **meaningful assertions on high-risk code,** and use coverage only to find *untested* risky areas.
9. **Flaky tests are bugs.** A test that sometimes fails for no reason teaches everyone to ignore failures. Fix it or quarantine it with a ticket and a deadline.
10. **Fast feedback first.** Run cheap checks on every commit and expensive ones less often (see the pipeline below).
11. **Say what you are NOT testing.** An explicit list of exclusions with reasons is part of an honest strategy.

A sensible pipeline (stages and frequency):

Stage	Runs	Contents	Target time
Every push	Automatically	Formatting, type checks, unit tests, secrets scan	Under 3 minutes
Every pull request	Automatically	Integration tests with real PostgreSQL, OpenAPI diff, dependency audit, migration-from-empty	Under 10 minutes
Merge to main	Automatically	Build image, deploy to staging, run migrations, **smoke tests**, API journey tests	Under 15 minutes
Nightly	Scheduled	Security scan of staging, short load test, data-quality probes, full migration-from-snapshot test	Unattended
Before each release	Triggered by the lead	Full load and soak test on a realistic dataset, restore drill (or evidence that one is recent), migration rehearsal on a production-size copy, exploratory charter, device matrix, manual security checklist	Hours

3.8 Release gates and evidence

A **release gate** is a **checkpoint with objective criteria** that must be met before work moves to the next stage. It turns "I feel ready" into "here is the proof."

A good gate criterion is:

- **Measurable.** "p95 save latency ⩽ 500 ms at 50 concurrent distinct users" not "performance is acceptable."
- **Binary.** Pass or fail, no "mostly."
- **Traceable.** It comes from a requirement (NFR) or a risk.
- **Owned.** A named person confirms it.
- **Backed by evidence.**

What counts as evidence? An **artifact that a third person could inspect later** and conclude the criterion was met. Examples:

Criterion type	Acceptable evidence
Automated tests pass	The CI run URL, the commit SHA, and the test report stored with the release
Performance meets target	The load-test report: configuration, dataset size, percentile table, error rate, database metrics
Restore works	The drill log: start and end times, the commands, row-count and checksum comparison, smoke test results
Security checks done	Scan reports, the filled checklist, the list of findings and their dispositions
Document exists	The runbook, committed, with a date and a reviewer
Residual risk accepted	A signed (or emailed, dated) acceptance by the risk owner with an expiry date

The two rules that separate real gates from theatre:

1. **Evidence is tied to the exact build.** A load test on last week's code proves nothing about this release candidate. Evidence records the **commit SHA or image digest** it applies to. If the code changes materially after the evidence was gathered, the evidence for the affected criteria is **stale** and must be re-run.
2. **A waiver is explicit, owned, and expires.** Sometimes a team will launch with a failed criterion. That is legitimate **only** if the owner of the affected risk accepts it *in writing*, states the residual risk plainly, and sets a date to re-check. **A silent waiver ("we'll fix it after launch") is how gates die.**

Gates in stages. Different checkpoints need different evidence:

Gate	When	Purpose	Typical evidence
G0: Merge gate	Every pull request	Keep the main branch healthy	CI green: unit, integration, contract diff, migration-from-empty, dependency audit
G1: Release-candidate gate	A build is proposed for release	Is this build good?	G0 plus staging deploy and smoke tests, API journeys, security scans, performance run, migration rehearsal, no open Sev1/Sev2 defects
G2: Go-live gate	Before production deployment or opening to users	Are we *operationally* ready?	G1 plus restore drill within RTO, rollback drill, monitoring and alert test, runbook reviewed, legal items (privacy policy, deletion flow), client sign-off on residual risks
G3: Post-release verification	First hours and days in production	Did it actually work in the real world?	Production smoke tests, dashboards stable during a watch period, data-quality probes clean, no rollback triggers hit

Defect severity needs a definition so the gate criterion "no open Sev1 or Sev2" means something:

Severity	Definition (aligned with the impact scale)	Gate rule
Sev1	Data loss/corruption, privacy exposure, wrong money shown broadly, or system down	Blocks release
Sev2	Major feature broken for many users; wrong data in an edge case; no workaround	Blocks release unless the owner accepts in writing
Sev3	Feature degraded with a workaround	Allowed; listed in release notes
Sev4	Cosmetic	Allowed

Rollback criteria are decided *before* release, while heads are cool. For example, for the first 48 hours: server error rate above 1% for 10 minutes; p95 save latency above 1 second for 15 minutes; any data-quality probe failing; any confirmed cross-user data exposure (immediate rollback and incident). Agreeing the numbers in advance prevents a 2 a.m. debate.

3.9 The traceability matrix

The artifact that holds everything together is a table that links:

```
Risk ID  →  Mitigation  →  Test(s)  →  Gate criterion  →  Evidence
```

Example row: *R-02 cross-user exposure → user-id filter in every repository query, 404 policy → two-user authorization tests for every endpoint + code review checklist → G1 "authorization tests pass for all endpoints" → CI run for commit abc123 and the checklist.*

The matrix answers the question "how do you know it won't?" for each important risk, and, just as usefully, reveals gaps: a risk with no test, or a test with no risk.

3.10 Keeping the register alive

A risk register is not a document you write once.

- **Review it on a schedule** (weekly during build, at every gate, and after every incident or near miss).
- **Add risks when things change:** a new feature, a new dependency, a new stakeholder demand.
- **Close risks explicitly** when the cause is gone or the event window has passed, and record why.
- **Learn from incidents:** after any incident, ask which risk it was, whether it was in the register, and whether the score was right. That is how your scoring improves.
- **Watch the trend:** the number of High and Critical risks should fall as you approach release. If it does not, you are accumulating risk, not managing it.

Part 4 — Practical demonstration: Spendwise, step by step

I'll show the weak approach, then build the strong one: the risk register, the scoring, the owners and responses, the test strategy, and the release gates.

4.1 The weak approach

```
RISKS:
 - Bugs
 - Security issues
 - Performance
 - Server might go down

TESTING PLAN:
 - Write unit tests for everything (target 100% coverage)
 - Test manually before launch
 - Use Selenium for UI tests

READY TO LAUNCH WHEN: all tests pass.
```

At least ten problems:

1. **"Bugs" is not a risk;** it is a certainty. No cause, event, or impact.
2. **No likelihood or impact,** so no ranking.
3. **No owners, no dates.**
4. **No mitigation or contingency.** What if the server goes down? "Fix it" is not a plan.
5. **Missing whole categories:** privacy, data recovery, store approval, bus factor, scope creep, legal.
6. **100% coverage as a target** rewards trivial tests and ignores whether anything is *checked*.
7. **Only unit tests.** The highest risks (constraints, concurrency, authorization, backups) live elsewhere.
8. **"Test manually"** is neither repeatable nor evidence.
9. **"All tests pass" is the entire gate,** with no performance, security, or recovery evidence.
10. **No definition of evidence, rollback criteria, or who decides.**

4.2 Step 1: gather the inputs

For the risk workshop I collect: the PRD (NFRs, exclusions, assumptions), the Technical Design (failure table, accepted risks, scale triggers), the ADRs (every "negative consequence" and "revisit if"), the API contracts (error cases, open questions), the dependency list, and the delivery plan.

4.3 Step 2: identify the risks (using several techniques)

Pre-mortem. "It is three months after launch and Spendwise failed. Why?" Answers I would write: *we lost people's data; somebody saw somebody else's expenses; the app was rejected by*

the store and we launched a month late; nobody used it; totals were wrong and people stopped trusting it; I was ill for two weeks and nothing could be fixed; someone found a key in the repository.

Walk the data flows. *Save expense:* retry duplicates it (R-04); a soft-deleted row is counted (R-05); a month boundary is off by one (R-05). *Sign-in:* credential stuffing (R-06); failures not counted because of rollback (R-06). *Refresh:* false reuse detection signs users out (R-12-ish). *Delete account:* data not purged (R-10).

Convert assumptions. "Users mostly online" → *if wrong, expenses fail to save in poor networks* (adoption risk R-15). "~10 users" → *if it becomes 100, capacity assumptions break* (R-14).

Read our own documents. ADR-001: single point of failure accepted → R-11. ADR-003: client-generated IDs → R-04. TDD §13: soft-delete filter forgotten → R-05; app-store review delays → R-08.

Threat model. Cross-user access → R-02; leaked secrets → R-03; sensitive data in logs → R-16.

Dependency and operations walk-through. App stores (R-08); cloud region or account problems (R-17); one engineer (R-07).

Category checklist. *Delivery:* scope creep (R-09). *Technical:* migrations (R-13), performance at volume (R-14). *Compliance:* deletion and retention (R-10).

Note how **different techniques found different risks.** The pre-mortem found the bus factor and the store; the data-flow walk found duplicates and wrong totals; the category checklist found scope creep. No single technique found them all.

4.4 Step 3: score them

The full register is below (section 4.9). Here is the scoring logic for a few, to show the reasoning, not just the numbers:

- **R-01 Unrestorable or slow backup. Likelihood 2, Impact 5 = 10.** *Why 2?* A managed database makes backups reliable, but *we have never restored one,* and untested restores fail more often than people expect. *Why 5?* If we cannot recover from data loss, that is permanent data loss for every user.
- **R-04 Duplicate or lost expense on a poor network. Likelihood 3, Impact 4 = 12.** *Why 3?* Mobile networks drop responses routinely, so the *trigger* is common, but idempotency should prevent the *effect. Why 4?* Wrong financial data shown.
- **R-05 Wrong totals (missed soft-delete filter; month boundaries). Likelihood 3, Impact 4 = 12.** *Silent failure,* so it deserves the extra band of attention and an automatic detector (data-quality probes and reconciliation tests).

- **R-17 Cloud region outage or provider account suspension. Likelihood 1, Impact 5 = 5 (Medium by score).** This is where the **impact-5 rule** applies: it is treated as High. The treatment is not to engineer multi-region for ten users, but to **record an owner-signed acceptance** with a contingency (an off-site backup copy and a documented rebuild procedure).

The heat map:

```
IMPACT
  5 |  R-17        R-01 R-02 R-03
  4 |              R-10  R-04 R-05 R-06 R-07 R-16
  3 |              R-13 R-14  R-11 R-12 R-15   R-08 R-09
  2 |
  1 |
    +-----------------------------------------------
       L=1          L=2        L=3              L=4        L=5
                         LIKELIHOOD
```

(Read: the top-right is worst. Nothing is Critical yet; seven risks sit in the High band, R-17 is promoted by the rule.)

4.5 Step 4: assign owners and review dates

In this project:

Owner	Risks (examples)	Why
Lead	R-01, R-02, R-03, R-04, R-05, R-06, R-11, R-12, R-13, R-14, R-16	Technical and operational controls
Meera (client)	R-08 (store accounts and privacy policy), R-09 (scope decisions), R-10 (legal retention obligations), R-15 (adoption), R-17 (accepting the residual risk)	Business, legal, and scope decisions
Lead, flagged as a risk to the project	R-07 (bus factor)	The lead is the single point of failure; mitigation is partly the client's (access to accounts, a second person with credentials in a sealed document)

Each risk gets a **review date** (weekly during build; at each gate).

4.6 Step 5: mitigation, early warning, and contingency for the top risks

Here are six risk "cards," the level of detail a High risk deserves.

R-01: Backup cannot be restored in time (Owner: Lead)

Element	Content
Statement	Because backups have never been restored, we may be unable to recover within RTO 8 h (and RPO 24 h) after data loss or corruption, causing an extended outage and possible permanent loss of users' expenses.
Inherent score	L2 × I5 = **10 (High; impact-5 rule)**
Mitigation (reduce / transfer)	Managed database with automated backups and point-in-time recovery; retention ⩽ 35 days; **one restore drill before go-live and quarterly**; infrastructure described as code or a step-by-step runbook; an extra encrypted export copy stored in a different account or region weekly
Early warning	Backup job failure alerts; backup age metric > 26 h; drill older than 90 days
Contingency	Trigger: data loss, corruption, or region outage. Action: follow the restore runbook; communicate to Meera within 30 minutes; restore to a new instance, verify, switch the connection string; post-incident review
Residual score	L1 × I5 = 5 after a successful drill (kept under review)
Evidence	Drill log with measured RTO; PITR test result; alert test record

R-02: Cross-user data exposure (Owner: Lead)

Element	Content
Statement	Because every query on user data must filter by user, a missed filter in any endpoint may expose one user's expenses to another, causing a breach of financial privacy.
Inherent score	L2 × I5 = **10 (High)**
Mitigation (reduce)	Identity only from the token; repository functions always take the user ID; `404` policy; code-review checklist item; evaluation of PostgreSQL Row-Level Security as defence in depth
Early warning	Any test failure in the authorization suite; unusual `404` rates on object endpoints
Contingency	Trigger: any confirmed exposure. Action: take the affected endpoint offline (feature flag or rollback), assess which users were exposed from logs, notify the client immediately and affected users as required, fix, add a regression test, review all similar endpoints
Residual	L1 × I5 = 5
Tests	Two-user authorization tests **for every object endpoint** (integration); a test that enumerates all routes and fails if a route has no authorization test (a cheap, powerful meta-test)
Evidence	CI report for the release commit; completed checklist

R-04: Duplicate or lost expense on a poor network (Owner: Lead)

Element	Content
Statement	Because mobile responses can be lost and users double-tap, an expense may be stored twice or appear lost, causing wrong totals and loss of trust.
Inherent	L3 × I4 = **12 (High)**
Mitigation	Client-generated UUID, primary key, `INSERT ... ON CONFLICT`, replay semantics (`200` / `201` / `409`), client retry policy with the same ID, button disabled while in flight
Early warning	A metric counting `200` replays and `409` conflicts; user reports
Contingency	Trigger: user reports a duplicate or a missing expense. Action: look up by `request_id`; confirm with the database; manual correction by support procedure; fix and add a regression test
Residual	L2 × I3 = 6
Tests	Unit: retry/backoff logic. Integration: concurrent identical creates → one row (statuses 201 and 200); retry after a simulated lost response; same ID with different content → `409`. Fault injection: crash after commit and before response → replay returns the row. E2E: airplane-mode journey on a real device
Evidence	CI report; fault-injection log; device test record

R-05: Wrong totals, a silent failure (Owner: Lead)

Element	Content
Statement	Because deleted rows must be excluded and months bounded in several queries, an error may show wrong totals while the app appears healthy.
Inherent	L3 × I4 = **12 (High; silent, so given extra attention)**
Mitigation	One repository read path for expense queries; `deleted_at IS NULL` in one place; `spent_on` as a calendar date; reconciliation tests
Early warning (detection is the point)	Nightly **data-quality probe:** recompute each sampled user's monthly total with an independent query (straight `SUM` over a fresh read) and compare with what the API returns; alert on any difference
Contingency	Trigger: probe mismatch or a user report. Action: disable the summary endpoint if needed; find the cause; correct the code; notify affected users
Residual	L2 × I3 = 6
Tests	Unit: month-boundary function (31st/1st, leap day, month lengths). Integration: create, delete, re-read totals; expense at the first and last day of months; export excludes deleted rows. Property-based: random create/edit/delete sequences vs a simple in-memory model
Evidence	CI report; probe history

R-08: App store review delay or rejection (Owner: Meera, with Lead support)

Element	Content
Statement	Because store review time varies and stores require specific items (privacy policy, in-app account deletion, accurate data disclosures), submission may be delayed or rejected, slipping the launch by days or weeks.
Inherent	L4 × I3 = **12 (High)**
Mitigation (avoid/reduce)	Submit an early test build (internal testing tracks) to find problems; implement account deletion and privacy policy first; review current store rules before submission; do not promise a date to the pilot group until approval; keep the backend backward-compatible so a delayed app does not block
Early warning	Review status; feedback from the test track
Contingency	Trigger: rejection or no decision by day 5. Action: respond to the reviewer with fixes; use direct distribution for testers where the platform allows; move the launch date and tell Meera early
Residual	L3 × I2 = 6
Evidence	Submission records; store checklist

R-07: Bus factor (Owner: Lead; flagged to Meera)

Element	Content
Statement	Because one engineer knows and controls everything, illness or unavailability may leave production issues unfixed for days.
Inherent	L3 × I4 = **12 (High)**
Mitigation	README and runbook; infrastructure as code or documented; credentials in a password manager with a sealed emergency copy held by Meera; clear incident procedure; a named backup contact
Early warning	Planned absences; runbook not updated after changes
Contingency	Trigger: lead unreachable for 24 h during an incident. Action: Meera follows the one-page runbook to switch to a maintenance page and contacts the backup engineer
Residual	L2 × I3 = 6
Evidence	A "dry run": someone other than the lead performs a deployment or restore using only the documents

4.7 Step 6: choose tests from the risks

A way to see the whole strategy is to look at which risk each test type serves. Reading across rows tells you *why each test exists:*

Risk	Unit	Integration	E2E	Security	Performance	Recovery
R-01 Backup						**Restore drill, PITR test, alert test**
R-02 Cross-user exposure		**Two-user tests, route meta-test**		**Authorization suite, code review**		
R-03 Secrets leaked				**Secrets scan, config check, log test**		Key-rotation drill
R-04 Duplicate/lost expense	Retry logic	**Idempotency, concurrency**	Airplane-mode journey			Fault injection
R-05 Wrong totals	**Month and money logic**	**Reconciliation, delete-then-read**				Data-quality probes
R-06 Credential attack	Throttle logic	**Failure counter persists, rate limit**		**Auth negative paths**		
R-11 Single instance outage			Smoke after restart		Soak test	**Dependency failure injection**
R-12 Old apps break		**OpenAPI diff, contract tests**	Old-version journey			
R-13 Migration locks		**Migration from snapshot**			**Rehearsal on prod-size copy**	Rollback drill

Risk	Unit	Integration	E2E	Security	Performance	Recovery
R-14 Performance at volume					**Load + soak on realistic data**	
R-16 Sensitive data in logs				**Log hygiene test**		

Notice two things. First, **every high risk is covered by tests at more than one level.** Second, **some risks are covered only by non-unit tests** (R-01, R-13, R-14), which is why a unit-test-only plan is not a strategy.

4.8 Step 7: write the test strategy

Key decisions I would write down, with reasons:

1. **Real PostgreSQL in CI**, same major version as production, for all repository, API, and migration tests. *Reason:* the critical guarantees live in the database; mocks would hide them.
2. **No SQLite shortcut.** *Reason:* SQLite lacks the features we rely on (partial indexes in our form, `ON CONFLICT` semantics, row-level locking, `timestamptz` behaviour), so tests could pass where production would fail.
3. **Few E2E tests** (five journeys) plus smoke tests after every deployment. *Reason:* wiring failures matter, flaky suites do not help.
4. **Performance tests on a seeded dataset of at least the 1,000-user estimate,** with **distinct simulated users,** against a production-like staging. *Reason:* otherwise the result is not evidence.
5. **Recovery drills are scheduled work,** not "if we have time." *Reason:* the biggest risks are only controlled by rehearsed contingencies.
6. **A two-user authorization test for every object endpoint,** enforced by a meta-test that lists all routes. *Reason:* BOLA is the most common and most damaging API flaw, and "remember to test it" is not reliable.
7. **Synthetic data only.** No production data in tests.
8. **Coverage is monitored for risk areas** (money, authorization, auth, deletion) rather than targeted globally.
9. **What we do not test (and why):** pixel-perfect layout on every device (low impact; manual spot check on the agreed matrix); multi-region failover (accepted risk, R-17); penetration testing by an outside specialist (recommended before opening beyond the

pilot, not for ten users; recorded as a revisit trigger); load beyond 1,000 users (outside the stated design envelope).

Examples of the tests themselves (sketches in the style of `pytest`; the exact fixtures depend on your setup):

A **unit test** for the threshold arithmetic: it uses integers and probes the boundaries.

```python
import pytest

@pytest.mark.parametrize("spent, limit, expected", [
    (3_999, 5_000, []),            # 79.98%  → no alert
    (4_000, 5_000, [80]),          # exactly 80% → alert (reach or exceed)
    (4_999, 5_000, [80]),
    (5_000, 5_000, [80, 100]),     # exactly 100%
    (3_000, 5_000, []),            # a jump is tested separately via the highest-t
    (0,     5_000, []),
])
def test_crossed_thresholds(spent, limit, expected):          # risk: R-05, FR-B3
    assert crossed_thresholds(spent_minor=spent, limit_minor=limit) == expected
```

An **integration test** against real PostgreSQL: a concurrency case that no mock could reproduce.

```python
import asyncio

async def test_concurrent_identical_creates_store_one_row(client, auth_headers, db
    payload = make_valid_expense()                                  # includes a c
    r1, r2 = await asyncio.gather(
        client.post("/v1/expenses", json=payload, headers=auth_headers),
        client.post("/v1/expenses", json=payload, headers=auth_headers),
    )
    assert sorted([r1.status_code, r2.status_code]) == [200, 201]   # one created,
    assert r1.json() == r2.json()                                   # same body
    count = await db.scalar(text("SELECT count(*) FROM expenses WHERE id = :id"),
                            {"id": payload["id"]})
    assert count == 1
```

A **constraint backstop test** that deliberately bypasses the API:

```python
import pytest
from sqlalchemy.exc import IntegrityError

async def test_database_rejects_zero_amount_even_without_the_api(db, a_user):   #
    with pytest.raises(IntegrityError):                                          # CHECK const
        await db.execute(text(
            "INSERT INTO expenses (id, user_id, category_id, amount_minor, spent_c
            "VALUES (:id, :uid, 1, 0, DATE '2026-10-12')"),
            {"id": new_uuid7(), "uid": a_user.id})
```

An **authorization test** (security):

```python
async def test_user_cannot_read_another_users_expense(client, headers_a, expense_c
    r = await client.get(f"/v1/expenses/{expense_of_b.id}", headers=headers_a)
    assert r.status_code == 404
    assert r.json()["code"] == "not_found"                    # same answer as a nor
```

A **meta-test** that stops you from forgetting an endpoint:

```python
def test_every_object_route_has_an_authorization_test(app, collected_authz_tests):
    object_routes = {(r.path, m) for r in app.routes if "{" in r.path for m in r.m
    missing = object_routes - collected_authz_tests
    assert not missing, f"Routes without a two-user authorization test: {missing}"
```

A **load-test script** (k6 style) that encodes the NFR as a pass/fail threshold:

```javascript
```

```
export const options = {
  scenarios: { saves: { executor: 'constant-vus', vus: 50, duration: '10m' } },  /
  thresholds: {
    http_req_duration: ['p(95)<500'],      // NFR-1: p95 <= 500 ms
    http_req_failed:   ['rate<0.001'],     // error rate below 0.1%
  },
};
// Each virtual user signs in as a DIFFERENT seeded user and creates expenses
// with a fresh UUID v7 per request. The database is pre-seeded to ~1,000 users of
```

A **recovery drill** is not code; it is a procedure with recorded numbers. An outline:

text

```
RESTORE DRILL — record start time T0
 1. Choose target: latest backup (RPO check) and a point-in-time T-2h (PITR check)
 2. Restore into a NEW database instance (never over production)
 3. Apply roles, extensions, and configuration per the runbook
 4. Point a staging API at the restored database; run migrations check (no pending
 5. Verify: table row counts vs source at that time; checksum of a sample
    (for example sum(amount_minor) per 20 sampled users); run smoke tests
 6. Record end time T1;   MEASURED RTO = T1 - T0;   note every manual step and sur
 7. Update the runbook with what was learned; attach the log as release evidence
```

4.9 Step 8: the sample risk register

Here is the register as it would be stored in the repository (Markdown table; sorted by score).

markdown

```
# Risk Register — Spendwise Pilot
**Version:** 0.1   **Date:** 2026-11-02   **Maintainer:** Tech Lead   **Next revie
**Scales:** Likelihood 1-5 (<5% ... >80%); Impact 1-5 (negligible ... severe).
**Bands:** 1-4 Low, 5-9 Medium, 10-14 High, 15-25 Critical.
**Rules:** impact 5 → at least High treatment; silent failures treated one band hi

| ID | Risk (cause → event → impact) | Cat. | L | I | Score | Owner | Response | M
|---|---|---|---|---|---|---|---|---|---|---|---|
| R-01 | Backups never restored → cannot recover within RTO/RPO → extended outage,
| R-02 | Missed user filter in a query → cross-user exposure → privacy breach | Se
| R-03 | Secret committed or shipped in app → key compromise → account takeover |
| R-04 | Lost response / double tap → duplicate or missing expense → wrong totals
| R-05 | Missed soft-delete filter / month boundary error → silently wrong totals
| R-06 | Credential stuffing / guessing → account takeover | Security | 3 | 4 | 12
| R-07 | Single engineer unavailable → production issues unfixed | People | 3 | 4
| R-08 | Store review delay/rejection → launch slips | External | 4 | 3 | 12 | Mee
| R-09 | Scope creep from stakeholders → delay and instability | Delivery | 4 | 3
| R-16 | Sensitive data (amounts, tokens) written to logs → privacy exposure | Sec
| R-17 | Region outage or provider account suspension → total outage | External |
| R-10 | Deletion job fails / backup retention misconfigured → data kept beyond pr
| R-11 | Single instance/DB fails → outage beyond 3.6 h/month | Ops | 3 | 3 | 9 |
| R-12 | API change breaks old app versions → app errors | Technical | 3 | 3 | 9 |
| R-15 | Users don't adopt logging habit → outcome not achieved | Product | 3 | 3
| R-13 | Migration locks tables or fails → outage | Technical | 2 | 3 | 6 | Lead |
| R-14 | Slow queries at data volume → NFR breach | Technical | 2 | 3 | 6 | Lead |
```

4.10 Step 9: the sample test strategy and release-gate documents

markdown

```
# Test Strategy — Spendwise (v0.1)
**Purpose:** gather evidence that high risks (Risk Register v0.1) are controlled b
**Principles:** risk-based depth; real PostgreSQL for DB-dependent behaviour; fail
on purpose; every test traces to a requirement or risk ID; synthetic data only.

## 1. Test levels
| Level | Purpose | Main risks | Runs | Pass evidence |
|---|---|---|---|---|
| Unit | Logic: money, dates, thresholds, policy, retry | R-04, R-05, R-06 | Every
| Integration (API + PostgreSQL) | Contract, idempotency, concurrency, constraints
| Contract diff | Protect old clients | R-12 | Every PR | OpenAPI diff clean |
| E2E + smoke | Wiring and deployment; 5 journeys | R-04, R-11 | Each staging depl
| Security | Authz, auth negatives, scans, config, logs | R-02, R-03, R-06, R-16 |
| Performance | NFR-1, capacity, soak | R-14, R-11 | Nightly short; pre-release fu
| Recovery | Restore, PITR, rollback, failure injection, key rotation | R-01, R-03
| Usability / compatibility / exploratory | NFR-2, device matrix, unknown unknowns
| Data-quality probes | Detect silent failures | R-05, R-10 | Nightly, in producti

## 2. Environments and data
Local; CI (ephemeral PostgreSQL, same major version as production); staging (produ
production (smoke + probes only). Synthetic data generators; performance dataset ≥
volume with distinct users; no production data in non-production.

## 3. Entry and exit criteria
Entry: build deployed to staging, migrations applied, smoke green.
Exit: release gates G1/G2 satisfied with evidence tied to the release commit.

## 4. Defects
Sev1-Sev4 as defined; Sev1 blocks; Sev2 blocks unless owner accepts in writing.

## 5. Not tested (and why)
Pixel-perfect layouts on all devices (low impact); multi-region failover (accepted
external penetration test (revisit before opening beyond pilot); load beyond 1,000

## 6. Risks to this strategy
Flaky tests (policy: fix or quarantine with deadline); single engineer (peer revie
test-data realism (review with each schema change).
```

markdown

```
# Release Gates — Spendwise (v0.1)
Evidence must reference the release commit SHA / image digest. Stale evidence is r

## G1 — Release candidate
| # | Criterion | Evidence | Source | Owner | Status |
|---|---|---|---|---|---|
| 1 | Unit + integration + contract tests pass | CI run URL, commit SHA | all | Le
| 2 | Two-user authorization tests exist and pass for every object route | CI repo
| 3 | OpenAPI diff shows no breaking change vs last release | Diff report | R-12 |
| 4 | Migrations apply from empty AND from previous-release snapshot with data | C
| 5 | Dependency audit: no known high/critical vulnerabilities (or accepted in wri
| 6 | Secrets scan clean (repo + history); log hygiene test passes | Reports | R-0
| 7 | Staging smoke + 5 journeys pass | Journey report | R-04, R-11 | Lead | |
| 8 | Load test: p95 save <= 500 ms and error rate < 0.1% at 50 concurrent DISTINC
| 9 | Soak test (2 h): no connection leak, stable latency | Report + DB metrics |
| 10 | Migration rehearsal on production-size copy: max lock wait < 3 s | Rehearsa
| 11 | No open Sev1/Sev2 defects | Defect list | all | Lead | |

## G2 — Go-live
| # | Criterion | Evidence | Source | Owner | Status |
|---|---|---|---|---|---|
| 12 | Restore drill completed; measured RTO <= 8 h; PITR to chosen time verified
| 13 | Rollback drill on staging completed; time recorded | Drill log | R-13 | Lea
| 14 | Alerts tested end-to-end (backup failure, error rate, uptime) | Alert test
| 15 | Runbook reviewed by a second person (dry run of deploy or restore) | Review
| 16 | Account deletion verified (live data removed; retention config <= 35 d); pr
| 17 | Store submission requirements met | Checklist | R-08 | Meera | |
| 18 | Residual risks reviewed and accepted by client (R-11, R-15, R-17) with expi
| 19 | Rollback criteria and 48-h watch plan agreed | Document | all | Lead + Meer

## G3 — Post-release (first 48 h)
Production smoke after deployment; dashboards reviewed at 1 h, 4 h, 24 h, 48 h;
data-quality probes clean; no rollback trigger hit.
Rollback triggers: 5xx > 1% for 10 min; p95 save > 1 s for 15 min; any probe failu
```

Part 5 — Documentation: what the lead records, and how much

Artifact	Purpose	Level of detail	Update rule
Risk register	The managed list of risks	One row per risk; a fuller "card" for each High risk	Weekly in build; at each gate; after incidents
Test strategy	How evidence is produced	3 to 5 pages: levels, environments, data, pipeline, exclusions	When risks or architecture change
Test plan for a release	What is run for *this* release	A short list drawn from the strategy	Per release
Traceability matrix	Risk → mitigation → test → gate → evidence	A table (can be part of the register)	Whenever risks or tests change
Release gate checklist	Go/no-go criteria with owners	One page per gate	Per release (copy the template, fill in evidence links)
Evidence archive	Reports, logs, drill records, sign-offs	One folder per release, named by commit SHA	Per release
Runbook	What to do when contingencies trigger	One page per scenario	After every drill and incident
Post-incident review	Learning	Timeline, cause, what worked, actions	After every incident

Practices that keep these alive:

- **Keep the register small and sharp.** Fifteen to twenty well-written risks beat eighty vague ones.
- **Attach IDs everywhere:** risk IDs in test names, gate criteria, and commit messages.
- **Store evidence automatically** where possible (CI artifacts), so gathering it is not a heroic effort.
- **Review the strategy after each incident:** the incident is data about what your tests missed.
- **Date every acceptance,** and give it an expiry.

Anti-patterns:

- **Shelfware register:** written once for a milestone, never opened again.
- **Scoring theatre:** long arguments about 9 versus 10, while no mitigation is actually done.
- **A coverage target as the quality goal.**
- **Tests that exist because "we always write them."** If a test has no risk, it has no reason.
- **Gates as rubber stamps:** every criterion ticked without evidence, or waived without an owner.
- **"Tested on my machine."** Evidence from a non-production-like environment without saying so.
- **Evidence for the wrong build.**

Part 6 — Trade-offs and failure scenarios

I mark each **[Established]** or **[Context].**

6.1 Qualitative matrix vs FMEA vs quantitative methods [Context]

A 5×5 matrix is quick and good enough for a small team. FMEA adds detectability and suits complex or safety-critical systems. Quantitative methods (expected loss in money) need data you rarely have. Choose the lightest method that changes decisions, and apply the corrections (impact-5 rule, silent failures).

6.2 Test pyramid vs heavier integration [Context]

Pyramid thinking guards against slow, flaky UI-heavy suites. But for a database-centred backend, real-database integration tests carry most of the value. Use the shape that matches *where your risks are.*

6.3 Mocking the database vs using a real one [Established for DB-dependent behaviour]

Mocks are fast and let you test logic in isolation, but they cannot reproduce constraints, locks, isolation, or `ON CONFLICT`. Use mocks (or no database) for pure logic; use a real PostgreSQL for anything involving persistence rules.

6.4 SQLite as a "faster" test database [Established]

A test suite that runs on SQLite while production is PostgreSQL will pass where production fails (partial indexes, locking, `timestamptz`, `ON CONFLICT` details, JSON behaviour). Same engine, same major version.

6.5 Too many end-to-end tests [Established]

E2E suites grow into slow, flaky pipelines that people stop trusting. Keep a handful of critical journeys and smoke tests; push everything else down a level.

6.6 Performance testing with unrealistic data or users [Established]

Scenario. A load test on an empty database passes at 30 ms. In production with three million rows, the same query takes 800 ms because no index matches. Or fifty simulated users who are all one user create artificial lock contention and a false alarm. **Prevention:** realistic volume, distinct users, refreshed statistics, production-like environment.

6.7 The backup that was never restored [Established]

Scenario. The backups run nightly for a year. On the day of a real incident, the restore fails because a role or extension is missing, or the backup is corrupt, or it takes 20 hours. **Prevention:** restore drills with measured times, repeated regularly.

6.8 Green CI, failed deployment [Established]

Scenario. All tests pass. The deployment fails because the migration locks a table on production-sized data, or an environment variable is missing. **Prevention:** migration rehearsal on a production-size copy; staging smoke tests; deployment checklist.

6.9 Testing in production [Context]

Synthetic monitoring, canary releases, and data-quality probes in production catch what pre-production cannot, but for financial data they must be read-only or use dedicated synthetic accounts, never real users' data. For a pilot, smoke tests and probes are enough; canaries become relevant at larger scale.

6.10 Chaos engineering [Context]

Deliberately breaking things in production is powerful at scale and premature for a pilot. A small version, **fault injection in staging** (stop the database, kill a process mid-request), delivers most of the learning at little risk.

6.11 Gates too strict, gates too lax [Context]

Too strict, and the project never ships, or people route around the gate. Too lax, and the gate is decoration. The calibration: **a criterion must be tied to a real risk or NFR, and a failed criterion must be fixable or consciously waived with an owner.**

6.12 The silent waiver [Established]

"We'll fix it after launch" without a record. Three weeks later nobody remembers it, or the person who agreed has left. **Prevention:** written, owned, dated waivers.

6.13 Stale evidence [Established]

A security scan from before a dependency upgrade; a load test from before an index change. **Prevention:** evidence tied to the release commit; re-run what a change could affect.

6.14 Risk register as paperwork [Established]

A beautiful register that nobody reviews, whose scores never change, and which did not predict the first incident. **Prevention:** weekly review, post-incident reconciliation ("was this in the register?"), and removing risks that are no longer real.

6.15 Coverage as a target [Established]

Teams write assertion-free tests to hit 90%. The number goes up and the confidence does not. Use coverage to find untested *risky* code, not to measure quality. *(Mutation testing, which checks whether tests notice injected bugs, is a stronger signal for critical modules, at higher cost.)*

6.16 Test data and privacy [Established]

Copying production data into staging "for realistic tests" moves personal financial data into a less protected environment. Use synthetic data; if a snapshot is unavoidable, anonymise and protect it as production.

6.17 Over-testing low-risk areas [Context]

Hours spent testing label colours while the restore path is unrehearsed. **Effort follows risk.**

Part 7 — Hands-on exercise

You are the lead. This is a realistic, messy situation, so read the inputs carefully; some of the evidence is flawed on purpose.

Situation

Release 1.1 (monthly category budgets with alerts, from earlier lectures) is scheduled to go live in **two weeks.** Since the pilot, Meera's marketing has worked: Spendwise now has about **200 active users** and roughly **40,000 expenses** stored. She wants to open sign-ups to the public **after** this release, so growth is expected.

What you know about the system:

- FastAPI service, **2 API processes,** each with a database connection pool of **10;** managed PostgreSQL (2 vCPU, 4 GB) with `max_connections` of 100; automated backups with PITR.
- Budget alerts are evaluated **inside the expense-write transaction** with a `SELECT ... FOR UPDATE` on the budget row and a unique constraint on the alerts table (ADR-006).
- The release adds two tables (`budgets`, `budget_alerts`) and one new index on the 40,000-row `expenses` table (the team has not yet decided how to create it).
- Push notifications were **excluded,** but Meera has already asked twice whether they can "just be added."

- It is just you, about **15 hours a week.** Meera has the app store accounts.

What has happened recently (incident history):

- Pilot week 3: a user on the metro reported **one expense appearing twice;** it was never reproduced.
- A tester reported the **80% alert shown twice** once.
- Last month an API deploy was rolled back by hand and took **45 minutes** because the steps were not written down.

Evidence gathered so far

Load test (staging, 200 users' worth of data: 40,000 expenses, 2 API processes, pool size 10 each). The test script used **5 test user accounts** shared among all virtual users:

Virtual users	Save p50	Save p95	Save p99	Error rate	DB CPU	Notes
10	60 ms	140 ms	220 ms	0%	12%	
25	85 ms	310 ms	540 ms	0%	25%	
50	190 ms	**900 ms**	2,100 ms	**0.8%** (all `503`, "pool timeout")	48%	~20 connections active; many sessions waiting on a lock on `budgets` rows

Restore drill (first ever, done last week by you, following notes in your head): total elapsed **11 hours.** Breakdown: provisioning and restoring the snapshot, 2 h; creating roles, extensions, and configuration by hand, 1 h 20 m; reconfiguring the API (connection strings, secrets), 1 h; DNS and TLS changes, 2 h; verifying the data by manually comparing counts table by table, 4 h 40 m. PITR to a specific timestamp was **not tested.** Your NFR-6 says RTO 8 h and RPO 24 h.

Tasks (about 2 to 2.5 hours)

1. **Identify risks (30 min).** Produce **at least twelve** risks for Release 1.1 as proper risk statements (cause, event, impact). State **which technique** (pre-mortem, data-flow walk, assumptions, documents, threat model, dependencies, category checklist) found each one; use at least four techniques. Include at least two **non-technical** risks and at least one **silent-failure** risk.
2. **Score (20 min).** Score each with the scales in section 3.3 and place them on a heat map. Justify **three** scores in a sentence each. Apply the impact-5 rule and the silent-failure rule

where relevant.

3. **Owners (10 min).** Assign an owner and a review date to each. Say which risks Meera owns and why.
4. **Responses (30 min).** For your **top five** risks: response type, mitigation, early warning, contingency with a trigger, residual score, and the tests or evidence that show the mitigation works.
5. **Test strategy (30 min).** For **each of unit, integration, end-to-end, security, performance, and recovery:** list at least three specific tests for this release, the risk each addresses, why that level is the right one, where and when it runs, and what evidence counts as a pass. List **three things you will deliberately not test** and why.
6. **Release gates (20 min).** Write the G1 and G2 checklists for this release: measurable criteria, evidence, owner. Define the **rollback criteria** for the first 48 hours with numbers.
7. **Read the evidence (25 min).** (a) **Analyse the load-test results.** What do they tell you, what do they *not* tell you, what flaws do you see in the test, what is the likely bottleneck, and what do you do next? Can you pass the gate? What would you change in the test? (b) **Analyse the restore drill.** Does it satisfy the gate criterion? Which parts of the 11 hours can be reduced and how? What is your decision, and what would a waiver require (and who may grant it)?
8. **Traceability (10 min).** A matrix row for each of your top five risks: risk → mitigation → test → gate criterion → evidence.

What I'll look for (so you know the standard)

- Risk statements with a cause, an event, and an impact (not "bugs").
- Scores that follow the scales and the correction rules.
- Owners who are one named person, with Meera owning business and legal risks.
- Mitigations that reduce likelihood or impact, and contingencies with *triggers*.
- Tests chosen **because of risks,** not by habit, with a reason for each level.
- Gate criteria that are **measurable and evidence-backed,** with evidence tied to the release build.
- A careful reading of the load-test data (a flaw in the test, not only in the system) and of the restore drill (a failed criterion, with a plan).
- An honest statement of what you are **not** testing.

Part 8 — Review of a typical attempt

As before, since this lecture is one-way, I review the kind of answer a learner at your level commonly writes: hints first, then corrections.

8.1 A typical first attempt (call the learner "Sam")

> **Risks:** 1. Bugs in budgets. 2. Security. 3. Performance. 4. Server down. 5. Users unhappy. 6. Deadline. **Scores:** all marked "High." **Owners:** "Team." **Mitigation:** "Test more. Fix bugs." **Tests:** "Unit tests with 100% coverage. Selenium tests for every screen." **Gate:** "All tests pass." **Load test:** "p95 is 900 ms because the server is too small. Upgrade the server and release." **Restore drill:** "11 hours is fine; RTO is only a guideline. Launch."

8.2 The review

What Sam did well.

- Sam covered several categories (product, delivery, security), not only code.
- Sam read the load-test table and noticed that latency rose with load, and correctly linked it to capacity as a hypothesis worth testing.
- Sam noticed that the restore drill took longer than the NFR, which is the first step to a correct decision.

Gap 1: the risks are labels, not statements. "Bugs in budgets" and "Security" give no cause, event, or impact, so nothing follows from them. *Hint:* Complete the sentence: *Because ___, ___ may happen, resulting in ___. Correction:* "Because alerts are evaluated inside the write transaction with a row lock, concurrent saves for the same budget may wait on each other, causing slow saves and pool exhaustion under load." And so on for each.

Gap 2: everything is "High." A ranking where everything is first ranks nothing. *Hint:* Which of your six risks would cause *permanent data loss* or *wrong financial data?* Which would only be annoying? *Correction:* score with the anchored scales; the restore and cross-user exposure risks are impact 5; "users unhappy" is a product risk with its own indicators, not a generic high.

Gap 3: the owner is "Team." Nobody is accountable. *Hint:* Who can *act* on the store accounts and on legal retention? Who can fix the code? *Correction:* one name each; Meera owns business and legal risks; the lead owns technical ones; both have review dates.

Gap 4: "Test more. Fix bugs" is not a mitigation. It has no mechanism and cannot be checked. *Hint:* What specifically reduces the *likelihood* or *impact* of a duplicate alert? What would you do if it happened anyway, and how would you know? *Correction:* mitigation (idempotent replay returns the same alerts, unique constraint); early warning (replay metrics); contingency with a trigger.

Gap 5: 100% coverage plus UI tests for every screen. This chases a number and builds a slow, flaky suite, while the highest risks (concurrency, constraints, authorization, backups, performance) are not tested at all. *Hint:* For each risk, which *kind* of test can fail if the risk is real? *Correction:* the risk-to-test mapping; a few E2E journeys; integration tests on real PostgreSQL; recovery drills.

Gap 6: the gate is "all tests pass." No performance, security, or recovery evidence; no definition of evidence; no owner; no rollback criteria. *Hint:* What would convince a skeptical third person? What build does the evidence belong to? *Correction:* measurable criteria with evidence tied to the commit, and rollback numbers.

Gap 7: the load-test conclusion ("server too small"). The data does not support it. Database CPU is only 48%, so the *database* is not saturated. The errors are **pool timeouts,** and many sessions are waiting on a lock on `budgets` rows. And the test used **5 user accounts** for 50 virtual users, so most requests contend for the same few budget rows, a contention pattern real users would not produce. *Hint:* What does "all 50 virtual users share 5 accounts" do to a design that locks a row per user and category? What does a pool of 10 per process mean for 50 concurrent requests? *Correction:* the test is **flawed,** so the result is not valid evidence in either direction. Fix the test (distinct users), re-run; examine pool sizing separately (see the reference).

Gap 8: "RTO is only a guideline." It is a stated NFR with a gate criterion. Also **PITR was never tested,** and the drill showed that knowledge lived in one person's head (the same cause as last month's slow rollback). *Hint:* Who may decide to accept a failed criterion? What would they need to be told? *Correction:* the gate fails; fix the drill process; a waiver, if any, belongs to Meera, in writing, with an expiry.

Overall verdict. Sam produced a list of worries and a ritual, not a risk-based strategy. The habit to build: **for every risk, name the evidence that would show it is controlled, and judge the evidence itself (is it valid, relevant to this build, and enough?) before trusting it.**

Part 8B — Reference solution to the exercise

Attempt the exercise first.

1 and 2. Risks and scores (Release 1.1)

ID	Risk (cause → event → impact)	Technique that found it	L	I	Score	Notes
N-01	Backups never restored in time (drill showed 11 h vs 8 h RTO; PITR untested) → recovery after data loss exceeds RTO → extended outage	Documents (NFR-6), observed evidence	2	5	10 (High; impact-5 rule)	Likelihood is that a restore is *needed* during the period; the *failure to meet RTO is already certain* (an observed gate failure)
N-02	A missed user filter in the new budget endpoints → cross-user exposure → privacy breach of financial data	Threat model	2	5	10 (High)	
N-03	Alerts evaluated under concurrency or retry → duplicate, missing, or wrong alerts → users distrust or disable alerts	Data-flow walk; incident history (duplicate alert seen)	3	3	9	Observed once, so L is at least 3
N-04	Wrong alert/status computation (thresholds, month boundary, soft-deleted rows) → users shown wrong financial information	Data-flow walk	3	4	12 (High; **silent**, so a probe is part of the mitigation)	
N-05	Row-lock contention and small connection pools (2 × 10) at 200+ users → latency and `503`s	Evidence (load test), design review (ADR-006 negatives)	3	3	9	Not yet proven; test flawed (see 7a)
N-06	New index and tables migrate in production → lock or failure → outage	Category checklist	2	4	8	40,000 rows is small, but growth and the first-ever production migration with real users raise it

ID	Risk (cause → event → impact)	Technique that found it	L	I	Score	Notes
N-07	Old app versions ignore the new `alerts` field or mishandle new responses → broken screens	Dependencies (old clients)	3	2	6	
N-08	App store review delay for the new version → launch slips	Dependency list	3	3	9	
N-09	Scope creep: push notifications "just added" → schedule and stability risk	Pre-mortem; incident history (asked twice)	4	3	12 (High)	Owner: Meera
N-10	Alert annoyance → users disable alerts or uninstall → product outcome missed	Assumptions (guardrail metric)	3	3	9	Product risk
N-11	Single engineer at 15 h/week with no written deploy/rollback steps → slow response to incidents	Pre-mortem; incident history (45-min rollback)	4	4	16 (**Critical**)	Observed; likelihood 4
N-12	Sensitive data (amounts, tokens) in logs → privacy exposure	Threat model	3	4	12 (High)	
N-13	Public sign-ups open after release → load and abuse grow faster than capacity assumptions (design envelope 1,000 users)	Assumptions (user count)	3	4	12 (High)	Scale triggers needed
N-14	Support load from 200+ users overwhelms a part-time engineer → slow fixes, errors from fatigue	Operational walk-through	3	3	9	
N-15	A vulnerable dependency or secret exposure →	Category checklist	3	3	9	

ID	Risk (cause → event → impact)	Technique that found it	L	I	Score	Notes
	compromise					
N-16	Credential stuffing as the user base grows publicly	Threat model	3	4	12 (High)	

(Observe the one **Critical** risk, N-11. It is not a code problem; it is the combination of one person, part-time hours, undocumented procedures, and an observed 45-minute rollback. A release blocker on a *people-and-process* risk is exactly the kind of thing a scoring scheme should be able to say.)

Justifying three scores: N-01: *impact 5* because inability to recover means permanent loss for every user; likelihood 2 because managed backups are reliable but never-restored backups fail often. N-04: *impact 4* (wrong financial data) and treated as **High and silent,** so the mitigation must include automatic detection. N-11: *likelihood 4* because it has *already happened* (the 45-minute rollback), so by the scale it cannot be "rare."

3. Owners and review dates

Owner	Risks	Reason
Lead	N-01, N-02, N-03, N-04, N-05, N-06, N-07, N-11, N-12, N-13, N-14, N-15, N-16	Technical and operational controls
Meera	N-08 (store accounts, submission), N-09 (scope decision), N-10 (product outcome), and **acceptance of any residual business risk**	Business, store, scope, and product decisions

Review: weekly until go-live, then at each gate; N-11 and N-01 reviewed every few days until closed.

4. Responses for the top five (N-11, N-01, N-04, N-02, N-13)

	N-11 Single engineer, no written procedures (Critical)	**N-01 Restore exceeds RTO (High)**
Response	Reduce	Reduce + Transfer
Mitigation	Write the deploy, rollback, and restore runbooks now; automate deploy and rollback to a single command; a second person (Meera or a contract engineer) performs a dry run using only the documents; a defined support window and an incident contact	Provisioning as code; scripted post-restore setup (roles, extensions, configuration); secrets and configuration prepared in advance for a standby environment; low DNS TTL and a health-checked endpoint; **automated verification script** (row counts and checksums) instead of 4 h 40 min of manual comparison; test PITR
Early warning	Rollback time measured in every release (target ⩽ 10 min); runbook last-reviewed date	Drill duration trend; backup-age alert; drill older than 90 days
Contingency	Trigger: lead unavailable for 24 h in an incident. Action: Meera follows the one-page runbook to a maintenance page and calls the backup engineer	Trigger: data loss or corruption. Action: restore runbook; client informed within 30 min
Residual	L2 × I3 = 6	L1 × I5 = 5 (after the re-drill meets ⩽ 5 h with margin)
Evidence	Rollback drill time; dry-run record signed by the second person	Second drill log with measured RTO; PITR test result

	N-04 Wrong alert/status (High, silent)	N-02 Cross-user exposure (High)	N-13 Public growth beyond assumptions (High)
Response	Reduce + Detect	Reduce	Reduce + monitor
Mitigation	Integer arithmetic; single read path; month derived from `spent_on`; reconciliation tests; **nightly probe** recomputing budget status independently	Identity from token only; user in every query; `404` policy; two-user tests for *every* budget route; route meta-test; code review checklist	Scale triggers (p95, DB CPU, pool saturation); capacity review before opening sign-ups; pool and `max_connections` sized with headroom; rate limits; public sign-up gated by a go/no-go review
Early warning	Probe mismatch; user reports	Authz test failure; abnormal `404` rate	Weekly active users trend; pool usage > 70%; p95 > 300 ms
Contingency	Trigger: mismatch. Action: disable alerts via a flag; fix; notify	Trigger: confirmed exposure. Action: disable endpoint, assess logs, notify client and users, fix, regression test	Trigger: any scale trigger. Action: scale instance; pause sign-ups
Residual	L2 × I3 = 6	L1 × I5 = 5	L2 × I3 = 6
Evidence	CI report; probe history	CI report incl. meta-test; checklist	Capacity review note; load report

5. Test strategy for Release 1.1

Level	At least three tests	Risk	Why this level	Runs	Evidence
Unit	Threshold function at boundaries (exactly 80%, 100%, jump over both); month-start derivation (31st/1st, leap day); highest-threshold selection rule	N-04	Pure logic; instant feedback; boundary cases are cheap here	Every push	CI report
Integration (real PostgreSQL)	Two concurrent saves crossing 80% → exactly one alert; retry after lost response returns the **same alerts** (linked by `triggered_by_expense_id`); unique constraint prevents duplicate alert even without the row lock; deleted expenses excluded from budget status; budget endpoints contract tests	N-03, N-04	Locks, unique constraints, and transactions exist only in the database	Every PR	CI report per commit
Migration	Apply from empty; apply to previous-release snapshot with data; rehearse on a production-size copy with lock timings; index creation done concurrently	N-06	Only real data reveals locks	PR + pre-release	CI + rehearsal log
Contract	OpenAPI diff clean; old-client fixture ignores `alerts`	N-07	Protects installed apps	Every PR	Diff report
End-to-end	API journey on staging: sign up → create budget → save expenses crossing 80% and 100% → see status → delete account; real-device journey on poor network (airplane mode toggled mid-save); post-deploy smoke	N-03, N-05	Catches wiring and deployment issues	Each staging deploy	Journey report; device notes
Security	Two-user tests for every budget route; route meta-test; secrets scan; dependency audit; log hygiene test	N-02, N-	Checks what the system *refuses* to do	PR + nightly	Reports; checklist

Level	At least three tests	Risk	Why this level	Runs	Evidence
	(no amounts or tokens); dynamic baseline scan of staging; rate-limit tests	12, N-15, N-16		+ pre-release	
Performance	**Redesigned load test:** 50 concurrent **distinct** users on a dataset of at least 1,000-user volume, with budgets for every user; pool-saturation test; 2-hour soak; a *contention test* that deliberately concentrates on one user to know the worst case	N-05, N-13	Only realistic load shows pool and lock behaviour	Nightly short; pre-release full	Load report with dataset, percentiles, DB metrics
Recovery	Second restore drill with measured RTO; PITR to a timestamp; rollback drill with timing; DB-stopped-in-staging failure injection; signing-key rotation drill	N-01, N-11	Contingencies are only real if rehearsed	Pre-release	Drill logs
Data-quality probes (production)	Recompute budget status for sampled users independently and compare; overdue deletions; expired-token backlog	N-04	Detect silent failures forever	Nightly	Probe history

Three things deliberately not tested: (1) push notifications (excluded; scope decision N-09); (2) load far beyond 1,000 users (outside the design envelope; covered by scale triggers and a pre-public-launch capacity review); (3) full visual regression across many devices (low impact; a manual spot check on the agreed device matrix).

6. Release gates for Release 1.1

G1 (release candidate):

#	Criterion	Evidence	Owner
1	All unit, integration, contract, migration-from-empty and migration-from-snapshot tests pass	CI run + commit SHA	Lead
2	Two-user authorization tests pass for every budget route; meta-test passes	CI report	Lead
3	Idempotent replay returns identical alerts; concurrent-save test passes	CI report	Lead
4	Dependency audit: no high/critical findings (or accepted in writing); secrets scan clean; log hygiene test passes	Reports	Lead
5	**Load test (valid design):** p95 save ⩽ 500 ms and errors < 0.1% at 50 concurrent distinct users on ⩾ 1,000-user dataset; no pool timeouts; DB CPU < 60%	Load report tied to the release SHA	Lead
6	Soak test 2 h: no connection growth, stable p95	Report	Lead
7	Migration rehearsal on production-size copy: lock wait < 3 s; index built concurrently and valid	Rehearsal log	Lead
8	No open Sev1/Sev2 defects	Defect list	Lead

G2 (go-live):

#	Criterion	Evidence	Owner
9	**Restore drill #2: measured RTO ⩽ 8 h (target ⩽ 5 h for margin); PITR verified**	Drill log with times and verification output	Lead
10	**Rollback drill: ⩽ 10 min,** using only the written runbook	Drill log	Lead
11	Dry run of deploy/rollback by a second person using only documents	Reviewer note	Lead + Meera
12	Alerts tested end-to-end; nightly probes running in production	Alert test record; probe output	Lead
13	Capacity review done before **public sign-ups** are opened; scale triggers configured	Review note	Lead + Meera
14	Store submission requirements met	Checklist	Meera
15	Residual risks reviewed and accepted by Meera (dated, with expiry)	Written acceptance	Meera

Rollback criteria for the first 48 hours: 5xx rate above 1% for 10 minutes; save p95 above 1 s for 15 minutes; any probe failure; pool usage above 90% for 10 minutes; any confirmed cross-user exposure (immediate rollback and incident); alert-disable rate above 30% in 24 hours triggers a *product* review (not a rollback).

7a. Reading the load-test evidence

What the data show:

- Latency grows nonlinearly with load: p95 140 → 310 → 900 ms. Errors appear only at 50 users and are all pool timeouts.
- Database CPU is only 48%, so the database engine is **not** CPU-saturated.
- About 20 connections are active, which is exactly **2 processes × pool of 10:** the pool is full. Requests beyond it wait, and some time out (the `503` s).
- Many sessions are waiting on locks on `budgets` rows.

What the data do *not* show:

- Whether this reflects realistic behaviour. The test used **5 accounts for 50 virtual users,** so roughly ten virtual users compete for *each* account's budget rows, serialised by

`SELECT ... FOR UPDATE`. Real users rarely save to the same budget at the same instant. The lock waits are probably an **artefact of the test.**

- Whether the pool size is too small *for realistic traffic.* The lock waits make each transaction last longer, which holds connections longer, which exhausts the pool sooner. Part of the pool problem is thus caused by the artificial contention.

What the likely explanation is: unrealistic concentration on a few budget rows causes lock waits; those lengthen transaction time; the pool of 20 connections saturates; requests queue and some fail. "The server is too small" is **not supported:** database CPU is half idle.

Can the gate pass? No, not on this evidence, and not because the system is proven bad: because the evidence is invalid. The criterion requires distinct users. So the correct status is *"G1 criterion 5 not met; evidence invalid; re-run required."*

What to do next:

1. **Fix the test:** at least 50 *distinct* seeded users, each with budgets; random think times; a mix of operations (saves, summaries). Seed a larger dataset (1,000-user volume) and refresh statistics.
2. **Keep the contention scenario too,** as a deliberate worst case (one user, many simultaneous saves, which is the double-tap or two-device case) and record what happens: it tells you how bad the lock design is under abuse, and whether a lock timeout and a clear `503` are needed.
3. **Examine the pool separately:** with 2 × 10 connections and 50 concurrent requests, queueing is expected unless each request holds a connection only briefly. Consider raising pool sizes within `max_connections` (for example 2 × 20 = 40 of 100, leaving headroom for migration, admin, and the purge job) or introducing a pooler later; set a sensible pool wait timeout and measure.
4. **Look inside the database during the run:** `pg_stat_activity` (waiting vs active), lock-wait events, `pg_stat_statements` for the budget queries, and `EXPLAIN (ANALYZE, BUFFERS)` of the sum query with the new index.
5. **Decision rule:** if the corrected test meets the criterion, the gate passes with evidence. If it still fails, then the lock scope or transaction length is a real problem: narrow the work inside the lock, and re-test.

7b. Reading the restore drill

Does it satisfy the criterion? No. Measured 11 h against RTO 8 h; also PITR was not tested, so RPO is unproven in practice. The gate criterion (G2-9) **fails.**

Where the time went, and what can be reduced:

Step	Actual	Improvement	Target
Provision and restore snapshot	2 h	Mostly provider time; start earlier in parallel with the next steps; pre-created standby configuration	~1.5 h
Roles, extensions, configuration by hand	1 h 20 m	Script it (idempotent SQL and configuration), keep it in the repository, test it	~10 min
Reconfigure API (connection strings, secrets)	1 h	Pre-defined environment for a "recovery" deployment; secrets referenced, not typed; one command	~15 min
DNS and TLS changes	2 h	Lower DNS TTL in advance; provision certificates ahead of time or use a provider-managed domain and certificate; document exact steps	~30 min
Verification by manual comparison	4 h 40 m	**Automate:** row counts and sampled checksums script; smoke tests against the restored database	~20 min
Total	**~11 h**		**~2.5 to 3 h**

(Note that the biggest reduction costs no money: it is the verification script. Writing it takes perhaps half a day, and it is also the evidence the gate requires.)

Decision: G2 does not pass yet. Fix and **re-drill** within the two weeks (it is feasible: a day of scripting, a half day of drill). Also test **PITR to a chosen timestamp** during that drill, and have a second person perform part of it from the runbook.

If the re-drill cannot be done in time, then launching would require a **waiver.** Who may grant it? **Meera,** because it is a business risk (a restore taking up to about eleven hours means the service could be down that long in a worst case, with no data lost if PITR works, which is itself unproven). A waiver would need: a plain-language statement of the residual risk ("in a data-loss incident, recovery may take up to about 11 hours, and recovery of the last day's data is unverified"), an **expiry date** (for example "re-drill within 14 days of launch"), named actions with dates, and a rule that **public sign-ups stay closed** until the criterion passes. My recommendation as lead would be to fix and re-drill rather than waive.

8. Traceability (top five)

Risk	Mitigation	Test(s)	Gate criterion	Evidence
N-11 Single engineer, no procedures	Runbooks; automated deploy/rollback; second-person dry run	Rollback drill; dry run	G2-10, G2-11	Drill log; reviewer note
N-01 Restore exceeds RTO	Scripted provisioning, verification script, pre-staged config	Restore drill #2; PITR test	G2-9	Drill log with measured RTO
N-04 Wrong alert/status	Integer arithmetic; single read path; nightly probe	Unit boundary tests; integration reconciliation; probe	G1-1, G1-3; G2-12	CI report; probe history
N-02 Cross-user exposure	Token identity; user in every query; `404`; meta-test	Two-user tests; meta-test	G1-2	CI report
N-13 Growth beyond assumptions	Scale triggers; capacity review; pool sizing	Redesigned load and soak; contention test	G1-5, G1-6; G2-13	Load report; review note

What to notice in the reference. Two pieces of *evidence* were wrong or insufficient, and the most valuable part of the exercise is **judging the evidence before trusting it:** the load test was flawed (so neither a pass nor a fail), and the restore drill failed its criterion and revealed a people-and-process risk. A lead's job is not only to run tests but to ask: *is this evidence valid, does it apply to this build, and does it actually answer the risk?*

Part 9 — Recall test, with answers

Cover the answers and try first.

Q1. Why is "testing exists to reduce risk by producing evidence" a better way to think than "testing finds bugs"? *Answer:* It ties every test to a specific risk and tells you what *not* to test. A test that reduces no identified risk is waste, and a high risk with no evidence is a gap. Finding bugs is one outcome; the purpose is knowing which risks are controlled.

Q2. Write a well-formed risk statement for "backups might not work." *Answer:* "Because backups have never been restored, we may be unable to recover within 8 hours after data loss, resulting in an extended outage and possible loss of users' data." (Cause, event, impact.)

Q3. Why is likelihood × impact misleading for a rare catastrophe, and what rule corrects it? *Answer:* A likelihood-1, impact-5 risk scores only 5 and looks Medium, but "rare" is a guess and the impact may end the project. Rule: any impact-5 risk receives at least High treatment, or an owner-signed acceptance with a contingency.

Q4. What is the difference between mitigation and contingency, and what four parts does a good contingency have? *Answer:* Mitigation acts before the event to reduce likelihood or impact; contingency is the plan if it happens anyway. A good contingency has a trigger, an early-warning indicator, a written action plan, and a rehearsal.

Q5. Why should the owner of a risk be one named person, and who owns business and legal risks? *Answer:* "The team" means nobody is accountable. The owner must be able to act or escalate. The client owns business, legal, and scope risks and accepts residual business risk; the lead owns technical and operational controls.

Q6. Why do we run integration tests against a real PostgreSQL rather than mocks or SQLite? *Answer:* Constraints, `ON CONFLICT`, row locks, transactions, isolation, and partial indexes exist only in the real database. Mocks or a different engine can pass where production fails.

Q7. Name three rules that make a performance test valid evidence. *Answer:* Realistic data volume with refreshed statistics; a realistic user distribution (distinct users, not a few shared accounts); a production-like environment and settings; percentiles rather than averages; watching the database during the run. (Any three.)

Q8. Why does a recovery drill count as a test, and what number does it produce? *Answer:* It rehearses a contingency; a plan never executed is an assumption. It produces the *measured RTO* (and, with point-in-time recovery, evidence for RPO), plus a corrected runbook.

Q9. What makes evidence valid for a release gate? *Answer:* It is inspectable by a third person, relevant to the criterion, and tied to the exact build (commit SHA or image digest). Evidence from a different build is stale and must be re-run.

Q10. What is a waiver, and what makes one legitimate? *Answer:* Launching despite a failed criterion. It is legitimate only if the owner of the affected risk accepts it in writing, the residual risk is stated plainly, and there is an expiry date and named follow-up actions. A silent "fix it after launch" is not a waiver.

Q11 (challenge). A teammate says: "Our CI is green and coverage is 95%, so we're safe to launch." Respond. *Answer:* Coverage measures which code ran, not what was checked; green CI shows only the tests that exist pass. Ask which risks those tests address: do they cover concurrency, constraints, authorization, migrations on real data, performance at volume, and recovery? Ask for the evidence for the criteria that CI cannot provide (restore drill, load test, rollback drill), tied to this build. Propose the traceability matrix to expose gaps.

Part 10 — Completion checklist

Tick honestly. Revisit the section noted for any you cannot.

- ☐ I can explain that **testing reduces risk by producing evidence,** and that a test without a risk is waste (3.1).
- ☐ I can write a **risk statement** with cause, event, and impact (3.1).
- ☐ I can identify risks with **several techniques:** pre-mortem, data-flow walk, assumptions, documents, threat model, dependencies, categories (3.2).
- ☐ I can define **anchored likelihood and impact scales** and explain **bands** (3.3).
- ☐ I can apply the **impact-5 rule** and the **silent-failure rule,** and explain residual versus inherent score (3.3).
- ☐ I can assign **owners** correctly, separating risk owners from action owners and business risks from technical ones (3.4).
- ☐ I can choose among **avoid, reduce, transfer, accept,** and write **mitigation, early warning, and contingency with a trigger** (3.5).
- ☐ I can explain **why each test type exists** and what it cannot catch: unit, integration, contract, E2E, security, performance, recovery (3.6).
- ☐ I can state **what makes a performance test valid** and **what a recovery drill proves** (3.6).
- ☐ I can design a **pipeline** with fast checks first and heavier checks later (3.7).
- ☐ I can write **release-gate criteria** that are measurable, binary, traceable, owned, and **evidence-backed,** with evidence tied to the build (3.8).
- ☐ I can define **severity levels, rollback criteria, and waiver rules** (3.8).
- ☐ I can build a **traceability matrix** from risk to evidence (3.9, 8B).
- ☐ I can **judge the quality of evidence** (spot a flawed load test; recognise a failed drill) (7, 8B).
- ☐ I know which choices are **context-dependent** and what would change them (6).

Closing: what you should understand before moving on

Before the next lecture, make sure you can say these five things in your own words:

1. **Testing is risk reduction through evidence.** Start from the risks, then choose the tests, not the other way round.
2. **A risk statement has a cause, an event, and an impact;** scoring ranks effort, but the corrections (impact 5, silent failures, observed failures) matter more than the arithmetic.
3. **Each test type has a reason and a blind spot.** Real-database integration tests, valid performance tests, and rehearsed recovery drills cover the risks that unit tests cannot.
4. **A gate is only as good as its evidence.** Evidence must be measurable, inspectable, tied to the exact build, and honestly judged, including noticing when the test itself is flawed.

5. **Contingencies must be rehearsed and waivers must be owned.** A plan never tried is a hope, and a failed criterion accepted without a name and a date is a silent waiver.

This is the end of Module 5, Lecture 1. I'm stopping here, as instructed.